Administration avancée de serveurs Linux

Travaux pratiques et projet intégrateur

GuillaumeYves
28 octobre 2026

Rapports techniques, configurations et résultats observés sur les VM du laboratoire.

TP1 : administration système avancée

Environnement

Exécution le 28 octobre 2026 sur `tp1-admin`, VM Ubuntu 24.04.5 LTS, noyau `6.8.0-147-generic`, 2 vCPU et 768 Mio de RAM. `systemd-detect-virt` retourne `kvm`.

Installation et commandes

Depuis la racine du dépôt :

```
python3 lab/manage.py start tp1-admin
python3 lab/manage.py wait tp1-admin
tar -cf - tp1 | python3 lab/manage.py ssh tp1-admin 'tar -xf -'
python3 lab/manage.py ssh tp1-admin 'sudo bash tp1/setup.sh'
python3 lab/manage.py ssh tp1-admin 'sudo bash tp1/verify.sh'
```

Le script installe `acl`, `cron`, `stress-ng` et `procps`, puis crée Alice, Bob et Carol si leurs comptes sont absents. Les modifications concernent `/srv/data`, `/backup`, `/usr/local/sbin/backup-data` et `/etc/cron.d/backup-data`.

Permissions

`/srv/data` appartient à root. Les ACL nominatives accordent `rwx` à Alice, `r-x` à Bob et `---` à Carol. Le droit `x` permet à Bob de traverser le répertoire pour lire les fichiers ; il ne lui donne pas le droit d'écrire. Le groupe propriétaire et les autres utilisateurs n'ont aucun accès. Les ACL par défaut transmettent ces règles aux nouveaux éléments. Les fichiers ordinaires créés sans droit d'exécution restent non exécutables.

```
setfacl -m u:alice:rwx,u:bob:r-x,u:carol:---,m::rwx /srv/data
setfacl -d -m u::rwx,u:alice:rwx,u:bob:r-x,u:carol:---,g::---,m::rwx,o::--- /srv/data
getfacl /srv/data
```

Alice a créé `check.txt`. Bob a lu son contenu ; sa tentative de création de fichier a retourné `Permission denied`. Carol n'a pas pu lister le répertoire. Ces refus sont les résultats attendus.

TP1 : getfacl /srv/data sur tp1-admin

```
# file: srv/data
# owner: root
# group: root
user::rwx
user:alice:rwx
user:bob:r-x
user:carol:---
group::---
mask::rwx
other::---
default:user::rwx
default:user:alice:rwx
default:user:bob:r-x
default:user:carol:---
default:group::---
default:mask::rwx
default:other::---

getfacl: Removing leading '/' from absolute path names
```

Sauvegarde

La tâche s'exécute quotidiennement à 02 h 00 dans le fuseau de la VM, UTC pendant le test. Le script utilise un verrou `flock` pour éviter les exécutions concurrentes. Il crée une archive temporaire, contrôle sa lisibilité, puis la renomme. `umask 077` et les permissions de `/backup` protègent les archives. `tar --acls --xattrs --numeric-owner` conserve les ACL, attributs et propriétaires.

Le contrôle a lancé la sauvegarde manuellement, extrait l'archive dans un répertoire temporaire et comparé le fichier avec `cmp`, puis les ACL avec `diff`. Les deux comparaisons ont réussi. Le service cron était actif. L'exécution à 02 h 00 n'a pas été attendue pendant cette séance ; sa configuration est livrée dans backup.cron.

La sauvegarde reste locale : une panne du disque de la VM pourrait supprimer données et archives. Une exploitation réelle demande aussi une copie sur un stockage distinct, une politique de rétention et des restaurations régulières.

Gestion de charge

La commande `top -b -n 1` a été exécutée avant et pendant `stress-ng --cpu 2 --timeout 15s`. Avant la charge, le CPU était inactif à 95,2 %. Pendant le test, chaque travailleur utilisait environ 100 % d'un vCPU.

`renice -n 10` a été appliqué au processus principal et aux deux travailleurs. `ps` a confirmé `NI=10`, et `top` a affiché `PR=30` et `NI=10`. Une valeur nice plus élevée réduit la priorité relative lorsque plusieurs processus se disputent le CPU. Elle ne fixe pas un plafond d'utilisation : les travailleurs ont donc continué à utiliser les CPU disponibles. Le processus de charge s'est terminé après 15 secondes.

Erreurs et résolution

Le premier amorçage du laboratoire a échoué pendant la création du compte initial `operator` par cloud-init. Ce compte a été remplacé par `labadmin` et la VM recréée. La nouvelle empreinte SSH a ensuite remplacé celle de cette VM jetable dans le fichier `known_hosts` propre au laboratoire. Le second amorçage a terminé avec `cloud-init status: done`.

Livrables et preuves

- Configuration et installation.
- Script de sauvegarde et tâche cron.
- Vérifications reproductibles.
- Sortie complète des vérifications.
- Sortie brute de getfacl et capture de cette sortie affichée dans un navigateur.

Les vérifications des accès, de la restauration et de la priorité ont réussi. Les fichiers sous `evidence/` proviennent de l'exécution dans la VM.

TP2 : sécurité et durcissement

Environnement et installation

Exécution le 28 octobre 2026 sur `tp2-security`, VM Ubuntu 24.04.5 LTS, 2 vCPU et 768 Mio de RAM. Adresse privée : `192.168.77.12`. Le TP1 sert de machine de test réseau, à l'adresse `192.168.77.11`.

```
python3 lab/manage.py start tp2-security
python3 lab/manage.py wait tp2-security
tar -cf - tp2 | python3 lab/manage.py ssh tp2-security 'tar -xf -'
python3 lab/manage.py ssh tp2-security 'sudo bash tp2/setup.sh'
python3 tp2/verify.py
```

Après l'installation, la connexion s'effectue avec `python3 lab/manage.py ssh tp2-security --user alice`. Le compte initial `labadmin` n'est plus autorisé en SSH. Le script peut être relancé par Alice avec `sudo` en fournissant le chemin de sa clé publique autorisée.

SSH

Le fichier complet sshd_config, commenté, remplace `/etc/ssh/sshd_config` après validation par `sshd -t`. L'original reste sauvegardé dans la VM. Il autorise uniquement Alice et exige une clé publique ; root, les mots de passe et l'authentification interactive sont désactivés. Les transferts de ports, d'agent et X11 sont désactivés. `MaxAuthTries 3` limite les essais par connexion.

La configuration effective a été contrôlée par `sshd -T`. Alice s'est connectée avec sa clé ; les connexions root et labadmin ont échoué avec `Permission denied (publickey)`.

Alice dispose de sudo sans mot de passe pour administrer ce laboratoire automatisé. La clé privée reste sur le poste de contrôle, dans `.lab/`, et n'est pas publiée.

Pare-feu

nftables.conf crée une table `inet`, couvrant IPv4 et IPv6. Les chaînes d'entrée et de transfert ont une politique `drop`. Les nouvelles connexions TCP sont limitées à SSH, HTTP et HTTPS. Le bouclage et les réponses aux connexions déjà établies sont acceptés. Une exception UDP ciblée sur la carte `ens3` conserve le DHCP nécessaire à l'accès sortant de cette VM. La sortie reste autorisée.

Pour distinguer un filtrage d'une absence de service, les tests ont démarré trois serveurs temporaires sur 80, 443 et 8088. Depuis le TP1, les connexions sur 22, 80 et 443 ont réussi ; 8088 est resté inaccessible. Une connexion locale a confirmé que le service 8088 écoutait bien. Les serveurs temporaires ont ensuite été arrêtés. Le test sur 443 vérifie l'ouverture TCP, sans déployer de certificat HTTPS dans cet exercice.

Les règles sont chargées par le service `nftables` au démarrage. Export réel du ruleset.

Fail2ban

sshd.local utilise le journal systemd et les actions nftables. Trois échecs dans une fenêtre de dix minutes entraînent un bannissement de dix minutes. Le bouclage et `10.0.2.2`, adresse de gestion du NAT QEMU, sont exemptés pour éviter de perdre le contrôle du laboratoire. Cette exemption doit être adaptée à l'adresse de gestion d'un environnement réel.

Les tentatives SSH d'un utilisateur inexistant depuis `192.168.77.11` ont produit trois échecs reconnus et un bannissement réel de cette adresse. `fail2ban-client status sshd` a affiché cette IP dans `Banned IP list`. Le test a ensuite levé le bannissement pour rendre le laboratoire réutilisable.

Audit

passwd.rules surveille les lectures, écritures et changements d'attributs sur `/etc/passwd`, avec la clé `passwd_watch`. Le fichier est installé dans `/etc/audit/rules.d/` puis chargé par `augenrules --load`.

Une lecture avec `cat /etc/passwd` a produit des événements `PATH` et `SYSCALL`, avec `syscall=openat`, `success=yes`, `exe=/usr/bin/cat` et `key=passwd_watch`. L'identité de connexion `auid=alice` reste visible même lorsque la commande est exécutée via sudo. Extrait réel des événements.

Erreur rencontrée

La première recherche `ausearch` lancée sans terminal via SSH ne retournait aucun événement, alors que le fichier d'audit en contenait. La lecture explicite avec `ausearch -if /var/log/audit/audit.log` a résolu le problème et a été intégrée au script de contrôle.

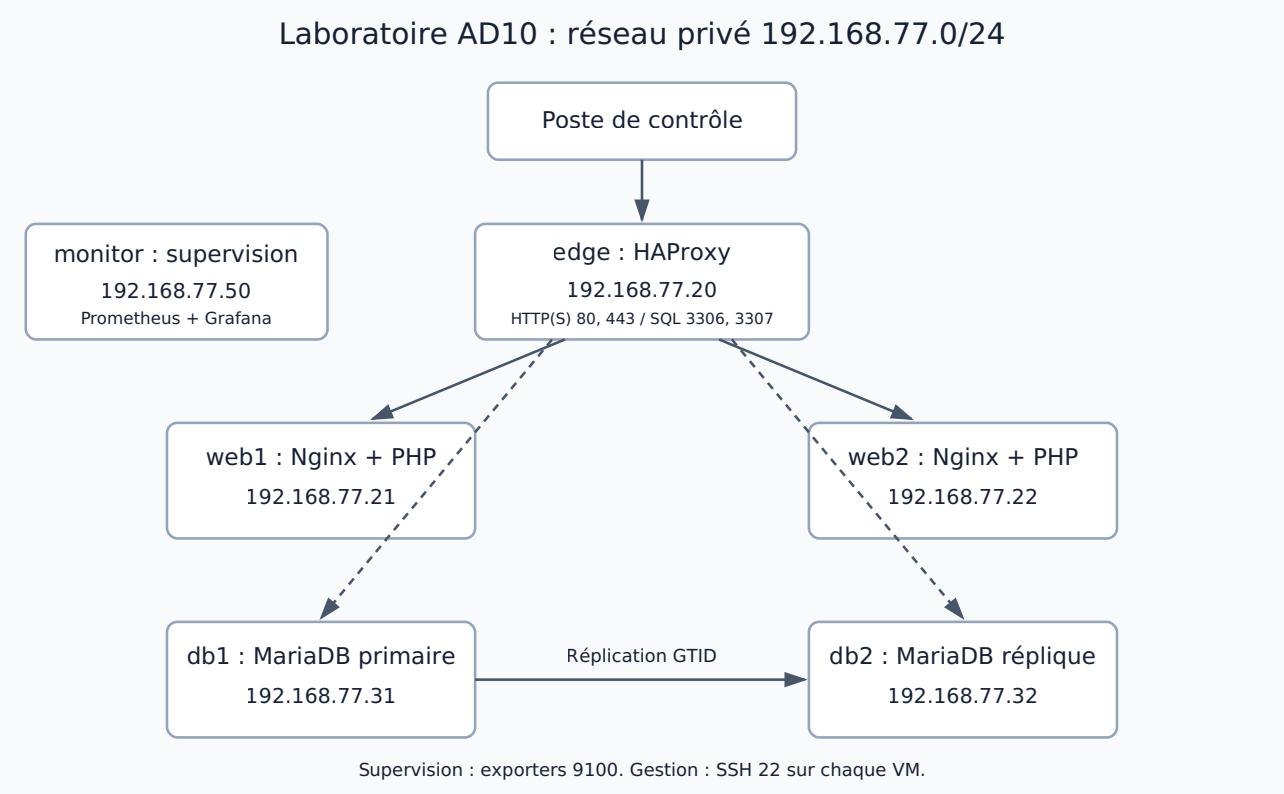
Résultat

Les quatre services `ssh`, `nftables`, `fail2ban` et `auditd` sont actifs. Les tests SSH, réseau, bannissement et audit ont réussi. Commandes et résultats complets.

TP3 : services réseau et haute disponibilité

Architecture

Le cluster comprend cinq VM Ubuntu 24.04.5 LTS : deux serveurs Nginx/PHP, deux serveurs MariaDB et un répartiteur HAProxy. Le réseau de service est distinct de la carte utilisée pour télécharger les paquets. Les contrôles ont été exécutés le 28 octobre 2026.



Le serveur de supervision représenté sur le schéma est ajouté au TP5, et HTTPS dans le projet final. Les échanges applicatifs passent par `edge`. Le port 3306 dirige les écritures vers le primaire ; 3307 dirige les lectures vers le primaire, puis vers la réplique s'il est indisponible.

Déploiement

```
python3 lab/manage.py start edge
python3 lab/manage.py start web1
python3 lab/manage.py start web2
python3 lab/manage.py start db1
python3 lab/manage.py start db2
python3 lab/generate_secrets.py
.venv/bin/ansible-galaxy collection install -r collections/requirements.yml
.venv/bin/ansible-playbook tp3/deploy.yml
python3 tp3/verify.py
```

Attendre la fin de cloud-init avec `python3 lab/manage.py wait NOM` avant le déploiement. Les commandes Ansible se lancent depuis la racine du dépôt. Inventaire, playbook et résultat du déploiement.

Les secrets aléatoires sont créés dans `.lab/secrets.json`, avec des permissions `0600`. Ils restent exclus de Git. Les tâches qui manipulent ces valeurs utilisent `no_log`. Le compte applicatif `shop_app` ne peut se connecter qu'à partir de l'adresse du proxy et possède les droits `SELECT`, `INSERT` et `UPDATE` sur `shop`.

Réplication MariaDB

Chaque serveur possède un identifiant unique : 31 pour db1 et 32 pour db2. Les journaux binaires utilisent le format ROW et la réplication utilise les GTID. Le compte `replication`, limité à l'adresse de db2, ne possède que `REPLICATION SLAVE`. La réplique utilise `read_only=ON`.

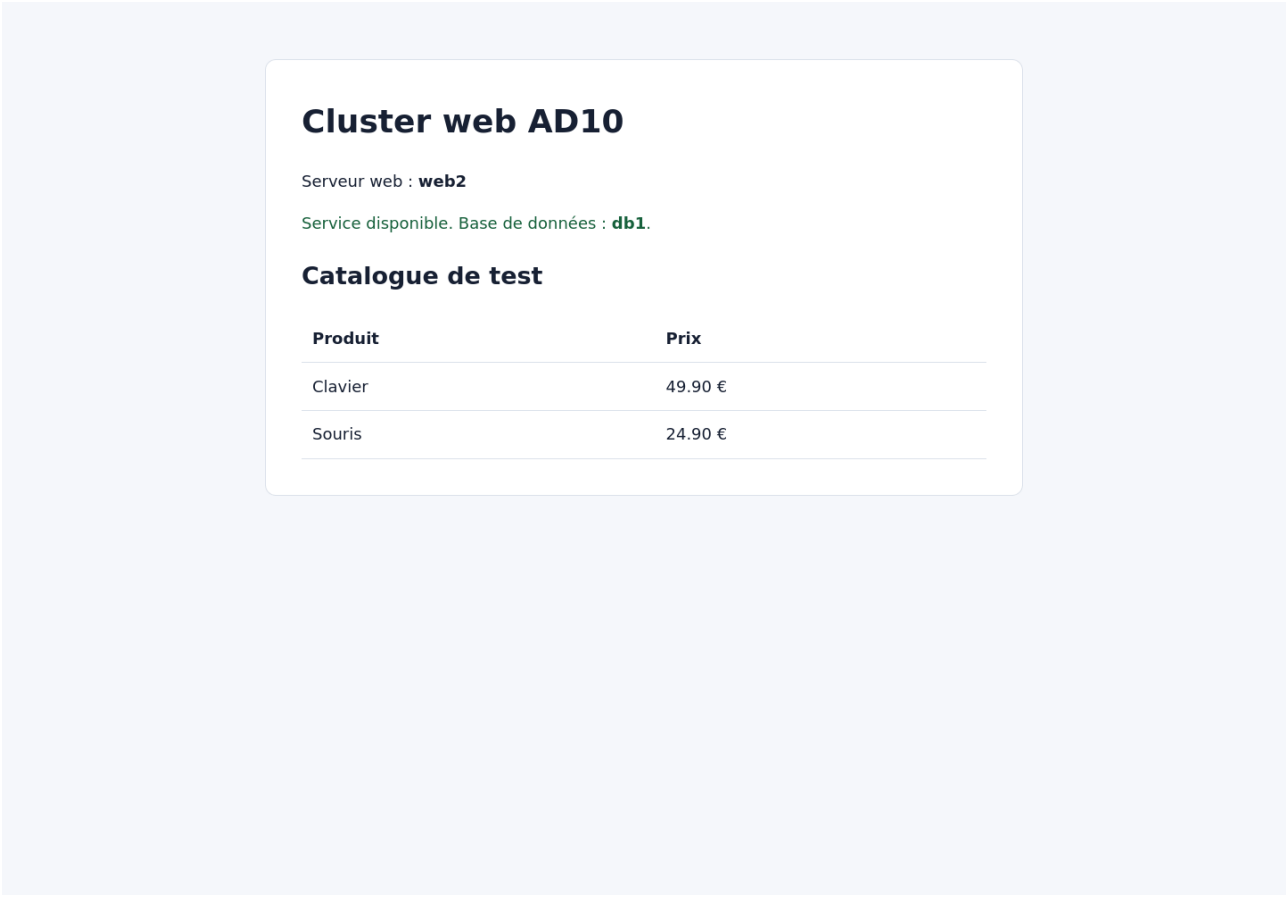
La réplication est démarrée avant la création du catalogue sur une installation neuve. Les deux bases ont reçu une valeur de contrôle unique ; sa présence sur db2 prouve la transmission des écritures. `SHOW SLAVE STATUS` a confirmé `Slave_IO_Running: Yes`, `Slave_SQL_Running: Yes` et `Seconds_Behind_Master: 0`.

Cette initialisation suppose une réplique neuve et des journaux binaires disponibles depuis la création des données. Pour réintégrer une ancienne base ou une réplique ayant perdu des journaux, il faut d'abord restaurer une sauvegarde cohérente. Le playbook ne réinitialise pas une réplication existante.

HAProxy et serveurs web

Le backend web utilise `roundrobin`. Les sondes HTTP sur `/health`, toutes les secondes, retirent un serveur après deux échecs et le réintègrent après deux réussites. La page PHP affiche le serveur web, la base interrogée et un catalogue lu dans MariaDB. Le fichier contenant le mot de passe applicatif se trouve hors de la racine web, dans `/etc/ad10-app.php`, lisible seulement par root et le groupe du service web.

La page est accessible sur localhost:8080. Le port est redirigé vers HAProxy ; il n'est pas exposé au réseau public. Configuration HAProxy réellement déployée.



Tests de résilience

Test	Résultat observé
12 requêtes en fonctionnement normal	Réponses des deux serveurs web, code HTTP 200.
Arrêt de Nginx sur web1	Les requêtes suivantes sont traitées par web2.
Retour de web1	Les deux serveurs participent à nouveau.
Arrêt de MariaDB sur db1	Le catalogue reste lisible depuis db2.
Lecture du mode de db2 pendant la panne	`read_only=1`, sans promotion automatique.
Retour de db1	Reprise des deux fils de réplication et du fonctionnement normal.

Le délai de reconnexion MariaDB par défaut est de 60 secondes. La reprise de réplication peut donc être plus lente que le retrait d'un serveur par HAProxy. Les fichiers de preuve contiennent les observations pendant cette attente.

La réplication est asynchrone : une écriture récente peut ne pas avoir atteint db2 lors d'une panne brutale. Les lectures peuvent continuer sur db2, mais les écritures nécessitent le primaire. Une réplication simple ne constitue donc pas une promotion automatique de base. HAProxy et son hôte restent aussi un point unique de panne dans ce laboratoire.

Erreurs corrigées

Le module Ansible installé accepte `primary_use_gtid: replica_pos` et le traduit pour MariaDB ; la valeur historique `slave_pos` a été remplacée. Le rechargement de Nginx a également été corrigé : la validation et le rechargement écoutent maintenant la même notification, car une validation sans changement ne déclenche pas un second handler.

Preuves

Sortie complète des tests. Les essais arrêtent réellement les services, puis les redémarrent dans un bloc de nettoyage pour remettre le laboratoire en état.

Références : réplication MariaDB, sondes HAProxy.

TP4 : automatisation et scripting

Machines et commandes

Deux VM dédiées, `tp4-web1` et `tp4-web2`, ont été utilisées le 28 octobre 2026. Elles sont distinctes du cluster du TP3.

```
python3 lab/manage.py start tp4-web1
python3 lab/manage.py start tp4-web2
python3 lab/manage.py wait tp4-web1
python3 lab/manage.py wait tp4-web2
tar -cf - tp4 | python3 lab/manage.py ssh tp4-web1 'tar -xf -'
python3 lab/manage.py ssh tp4-web1 'sudo bash tp4/install-web.sh'
.venv/bin/ansible-playbook -i tp4/inventory.ini tp4/playbook.yml
.venv/bin/ansible-playbook -i tp4/inventory.ini tp4/playbook.yml
python3 tp4/verify.py
```

Le script Bash a d'abord été exécuté sur le premier serveur. Le playbook a ensuite été exécuté sur les deux serveurs dans un même play, avec cinq connexions parallèles disponibles (`forks = 5`). Les lignes intercalées des deux hôtes dans les journaux montrent leur prise en charge pendant la même exécution.

Script Bash

install-web.sh vérifie les droits root, installe Nginx, copie index.html, crée le site, retire le site par défaut, valide la configuration, active et démarre le service. Les erreurs interrompent l'exécution avec `set -euo pipefail`.

Les fichiers modifiés sont `/var/www/ad10-static/index.html`, `/etc/nginx/sites-available/ad10-static` et le lien dans `sites-enabled`.

Playbook et rôle de pare-feu

playbook.yml exprime le même état avec les modules `apt`, `file`, `copy`, `template` et `service`. Le rôle static_web gère Nginx. Les modifications de configuration déclenchent une validation, puis un rechargement, par handlers.

Le rôle réutilisable firewall installe nftables, valide les règles avant leur installation et active le service au démarrage. Les entrées sont bloquées par défaut. SSH et HTTP sont autorisés ; le bouclage, les réponses aux connexions et le DHCP de la carte de sortie sont préservés. Le rôle accepte aussi des règles limitées à des adresses sources pour le projet final.

Comparaison

Critère	Bash	Ansible
Description	Suite de commandes à exécuter.	État attendu des ressources.

Critère	Bash	Ansible
Exécution distante	Nécessite une connexion et une copie explicites.	Inventaire SSH et exécution sur plusieurs hôtes.
Réexécution	Réinstalle ou réécrit selon la logique du script.	Ne modifie que les ressources différentes.
Redémarrage	Appels explicites à systemctl.	Handlers déclenchés par les modifications.
Erreurs	Arrêt du script et sortie de commande.	Résultat détaillé par tâche et par hôte.
Organisation	Adapté à une installation courte.	Rôles, variables et inventaires réutilisables.

L'idempotence dépend du contenu des tâches ; Ansible ne la garantit pas pour une commande arbitraire. Ici, les modules déclaratifs permettent une seconde exécution avec `changed=0` sur les deux machines.

Vérifications et résultats

Les deux pages ont répondu en HTTP 200 sur localhost:8041 et localhost:8042. L'en-tête `X-Backend` distingue les deux machines. Nginx et nftables sont actifs et activés au démarrage. Le contrôle de la seconde exécution confirme `changed=0`, `unreachable=0` et `failed=0` pour chaque hôte.

- Exécution réelle du script Bash.
- Première exécution Ansible.
- Seconde exécution Ansible.
- Résultats du contrôle.

Aucune erreur d'exécution n'a été observée sur ces deux VM pendant le déploiement du TP4.

TP5 : supervision et monitoring

Installation

Une VM dédiée, `monitor` (`192.168.77.50`, 2 vCPU, 2 Gio de RAM), héberge Prometheus, Grafana et Alertmanager. Les serveurs web du TP3 exposent leurs mesures avec Node Exporter sur le réseau privé. Les tests ont été réalisés le 28 octobre 2026.

```
python3 lab/manage.py start monitor
python3 lab/manage.py wait monitor
.venv/bin/ansible-playbook tp5/deploy.yml
.venv/bin/python tp5/verify.py
```

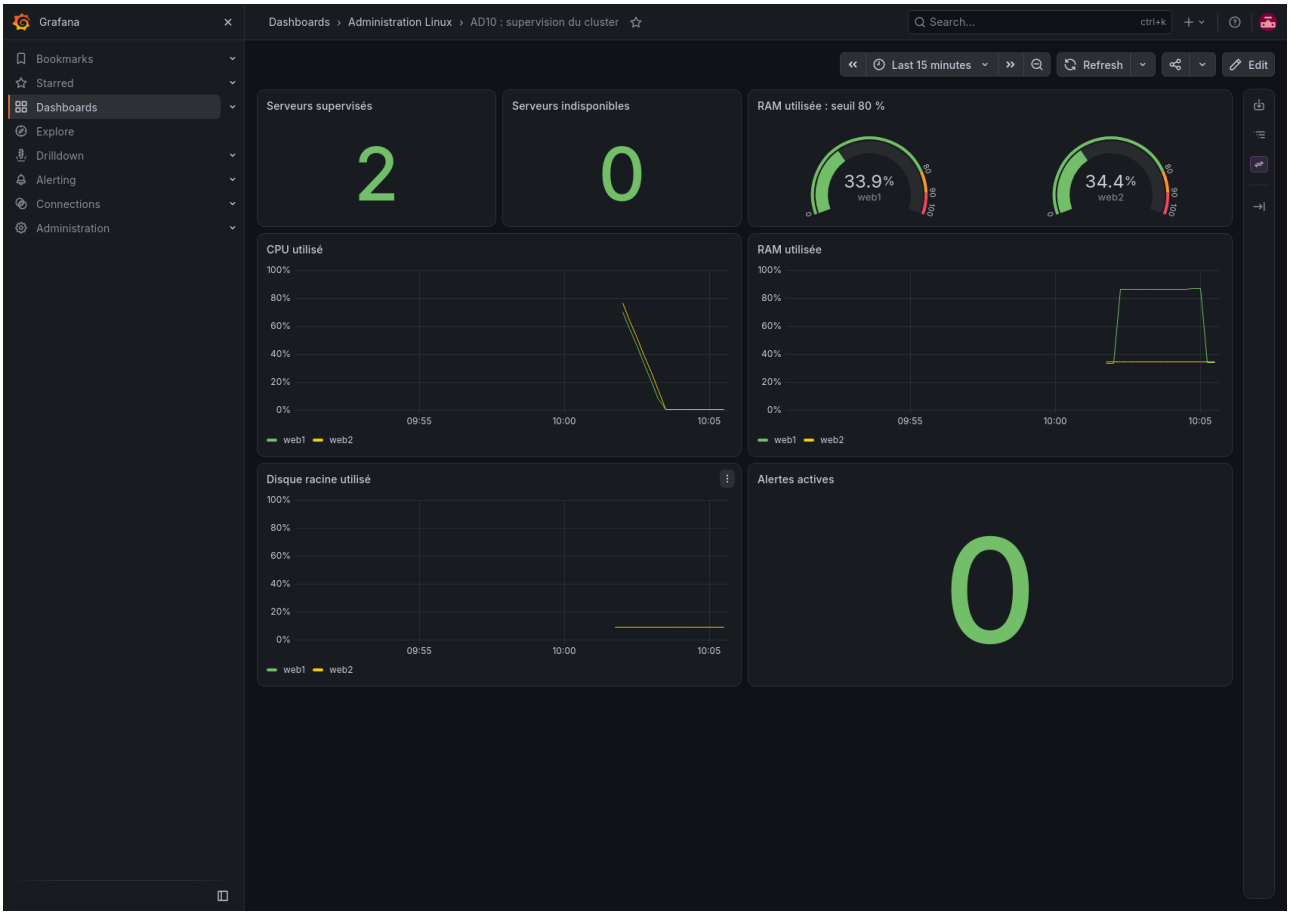
Le rôle utilise les paquets Ubuntu pour Prometheus, Alertmanager et Node Exporter. Grafana OSS provient de son dépôt APT officiel, avec une clé dédiée et `signed-by`. Les versions réellement installées figurent à la fin du journal de vérification.

Collecte et tableau de bord

Configuration Prometheus. Les mesures sont collectées toutes les 15 secondes ; les règles sont aussi évaluées toutes les 15 secondes. La rétention locale est de sept jours. Les cibles web1 et web2 possèdent une étiquette `host` pour faciliter les comparaisons.

Grafana est provisionné par fichiers : une source Prometheus et un tableau de bord personnalisé à sept panneaux. Il affiche le nombre de serveurs disponibles et indisponibles, les pourcentages CPU, RAM et disque, ainsi que les alertes actives. Les requêtes CPU utilisent `rate` sur deux minutes ; quelques échantillons sont donc nécessaires après le démarrage.

Les interfaces locales sont Grafana, Prometheus et Alertmanager. Grafana exige une connexion avec le compte `admin` ; son mot de passe aléatoire est conservé dans `.lab/secrets.json` et n'est pas publié. Le tableau de bord n'autorise pas l'accès anonyme.



Stratégie d'alerte mémoire

La règle MemoryUsageHigh utilise :

```
(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100 > 80
```

`MemAvailable` tient compte de la mémoire récupérable par Linux ; cette mesure évite de considérer tout le cache comme une consommation irréversible. Le seuil est strictement supérieur à 80 %. Le délai `for: 2m` filtre les pointes courtes. L'état devient `pending`, puis `firing` si le dépassement dure deux minutes. Il se résout quand la condition cesse d'être vraie.

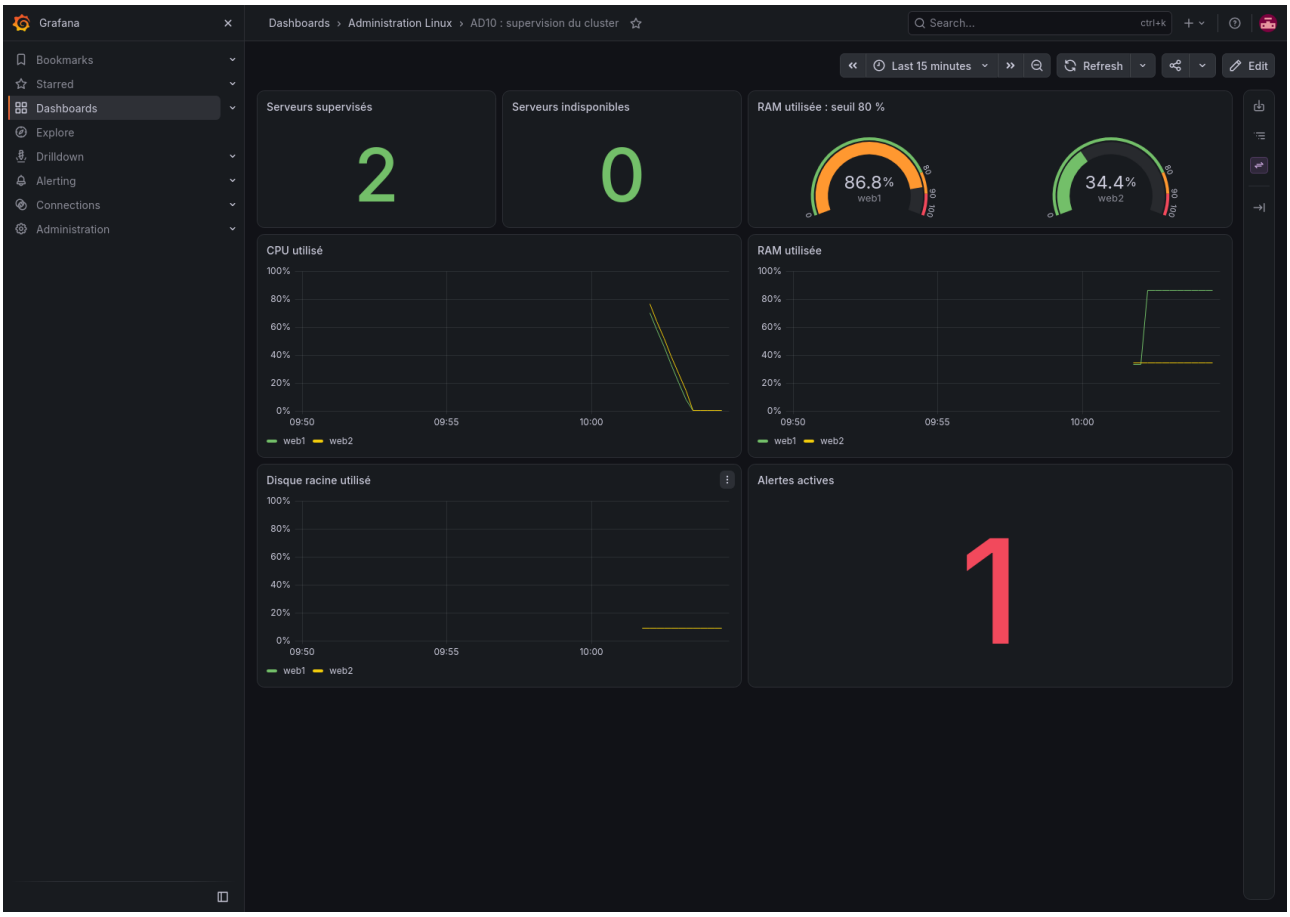
Alertmanager groupe les événements par nom d'alerte et serveur, attend cinq secondes avant la première notification et répète une alerte persistante après une heure. Le récepteur HTTP local enregistre les notifications dans journald, y compris les résolutions. Aucun destinataire externe n'est configuré dans le laboratoire. Une exploitation réelle doit raccorder ce récepteur à la chaîne d'astreinte de l'équipe.

Une seconde règle, `NodeUnavailable`, détecte la perte d'un exporteur pendant une minute.

Vérifications

Les tests `promtool` vérifient quatre situations : 80 % sans alerte, 85 % pendant moins de deux minutes sans notification active, 85 % pendant deux minutes avec alerte, puis retour sous le seuil avec résolution. Les tests ont réussi.

Un essai réel a ensuite alloué et touché des pages mémoire sur web1 pour laisser environ 13 % de mémoire disponible. Le processus était limité à 600 Mio par systemd et à 250 secondes d'exécution. Prometheus a observé les états `pending` puis `firing`. Alertmanager a reçu l'événement, et la capture ci-dessous a été prise pendant le dépassement.



Le processus de test a été arrêté dans un bloc de nettoyage. Après libération de la mémoire, l'alerte s'est résolue. Le journal du récepteur confirme les notifications `firing` et `resolved`.

Livrables

- Playbook, rôle de supervision et rôle Node Exporter.
- Déploiement réel et vérifications complètes.
- Cibles collectées.
- Alerte active Prometheus et réception Alertmanager.
- Notifications de déclenchement et de résolution.

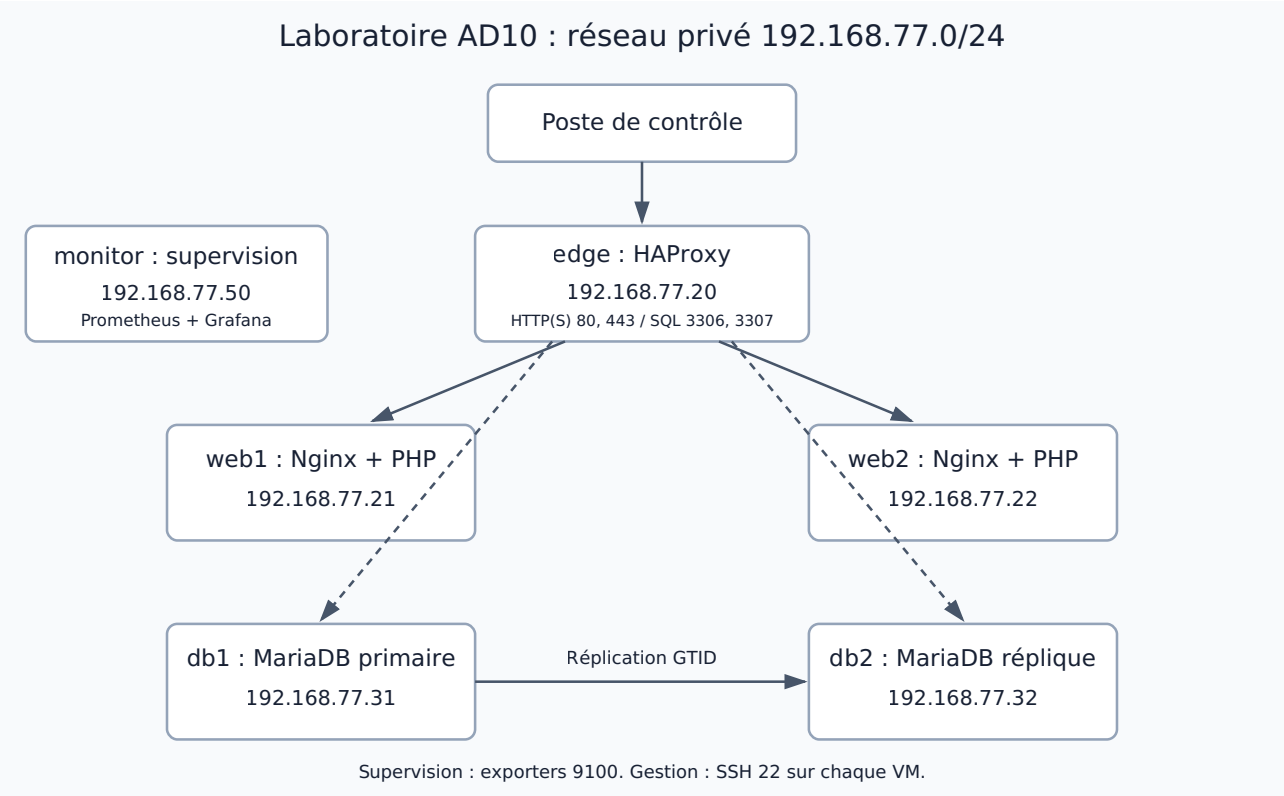
Aucune erreur n'a été observée pendant le déploiement de la supervision. Un avertissement Ansible a signalé la création de son répertoire temporaire root avec des permissions `0700` ; le déploiement a continué normalement.

Références : installation officielle Grafana, provisionnement Grafana, règles d'alerte Prometheus.

Projet final : infrastructure sécurisée, automatisée et supervisée

Périmètre

Le laboratoire héberge un catalogue de test sur deux serveurs web, une base MariaDB répliquée, un répartiteur HAProxy et une plateforme Prometheus/Grafana. Les six serveurs sont des VM Ubuntu dédiées. Les configurations sont déployées par Ansible et les résultats ci-dessous sont vérifiés sur l'infrastructure réelle.



Machines et ressources

Machine	Adresse privée	Rôle	RAM
edge	192.168.77.20	HAProxy, terminaison HTTPS et proxy SQL.	768 Mio
web1	192.168.77.21	Nginx et PHP-FPM.	768 Mio
web2	192.168.77.22	Nginx et PHP-FPM.	768 Mio
db1	192.168.77.31	MariaDB primaire et sauvegardes.	1 Gio
db2	192.168.77.32	MariaDB réplique en lecture seule.	1 Gio
monitor	192.168.77.50	Prometheus, Grafana et Alertmanager.	2 Gio

Chaque VM utilise deux vCPU et un disque extensible de 12 Gio. Les disques sont des images différentielles : seules les modifications consomment de l'espace en plus de l'image Ubuntu commune. Les machines des TP1, TP2 et TP4 restent distinctes du cluster.

Installation depuis un dépôt neuf

L'hôte Linux doit disposer de KVM et l'utilisateur doit avoir accès à `/dev/kvm`. Pour l'exécution depuis Windows, lancer ces commandes dans Ubuntu/WSL. Sur un autre hyperviseur, adapter l'inventaire aux adresses SSH des VM, ainsi que les interfaces `ens3`, `wan` et `lan` utilisées par le réseau et le pare-feu.

```
sudo apt-get update
sudo apt-get install qemu-system-x86 qemu-utils cloud-image-utils python3-venv
sudo usermod -aG kvm "$USER"
# Ouvrir une nouvelle session pour prendre en compte le groupe.
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/ansible-galaxy collection install -r collections/requirements.yml
.venv/bin/playwright install firefox
sudo .venv/bin/playwright install-deps firefox

python3 lab/manage.py start edge
python3 lab/manage.py start web1
python3 lab/manage.py start web2
python3 lab/manage.py start db1
python3 lab/manage.py start db2
python3 lab/manage.py start monitor
python3 lab/manage.py wait edge
python3 lab/manage.py wait web1
python3 lab/manage.py wait web2
python3 lab/manage.py wait db1
python3 lab/manage.py wait db2
python3 lab/manage.py wait monitor

python3 lab/generate_secrets.py
python3 lab/create_tls.py
.venv/bin/ansible-playbook final/bootstrap.yml
.venv/bin/ansible-playbook final/site.yml -e ansible_user=alice
.venv/bin/python final/verify.py
```

Les chemins des fichiers supposent une exécution depuis la racine du dépôt. `bootstrap.yml` prépare Alice avec le compte initial `labadmin`. Le déploiement principal utilise ensuite Alice et lui réserve l'accès SSH. Pour les réexécutions, utiliser directement `final/site.yml -e ansible_user=alice`.

Le laboratoire télécharge une image Ubuntu officielle et compare sa somme SHA-256 à celle du serveur de publication. Les clés SSH, certificats privés, secrets et disques sont conservés dans `.lab/`, avec des permissions restrictives, et sont exclus de Git. Ne pas supprimer les secrets existants pendant une simple réexécution : cela changerait les identifiants attendus.

Accès et chiffrement

Usage	Adresse locale	Authentification
Catalogue	https://localhost:8443	Lecture publique de données de test.
Grafana	http://localhost:3000	Compte admin, secret dans `.lab/secrets.json`.
Prometheus	http://localhost:9090	Accès limité au poste de gestion par le pare-feu.
Alertmanager	http://localhost:9093	Accès limité au poste de gestion par le pare-feu.
SSH	`python3 lab/manage.py ssh NOM --user alice`	Clé privée locale et compte Alice.

Le port HTTP 8080 redirige vers HTTPS 8443. Le certificat est autosigné pour `localhost`, `127.0.0.1` et l'adresse privée de HAProxy ; le navigateur demande donc de lui faire confiance. Le contrôle automatisé utilise explicitement ce certificat comme autorité de confiance et vérifie l'identité du serveur, sans désactiver la validation TLS.

```
curl --cacert .lab/tls/edge.crt https://localhost:8443/
```

HAProxy impose TLS 1.2 au minimum. Les interfaces de supervision en HTTP ne sont exposées que sur les redirections locales du laboratoire ; pour les administrer depuis un autre poste, utiliser un tunnel sécurisé ou ajouter une terminaison TLS adaptée. Le certificat local doit être remplacé par un certificat reconnu pour un déploiement public.

Sécurité réseau

Toutes les machines appliquent une politique d'entrée et de transfert `drop`. Les connexions établies, le bouclage et le DHCP de la carte de sortie sont préservés. Les flux de service sont limités par port et adresse source.

Destination	Ports TCP	Sources autorisées
Tous les serveurs	22	Gestion SSH par clé, compte Alice uniquement.
edge	80, 443	Clients du laboratoire.
web1, web2	80	edge uniquement.
db1, db2	3306	edge et serveur DB pair.
edge	3306, 3307	web1 et web2 uniquement.
Tous les exporteurs	9100	monitor uniquement.

Destination	Ports TCP	Sources autorisées
monitor	3000, 9090, 9093	Adresse de gestion du NAT QEMU, `10.0.2.2`.

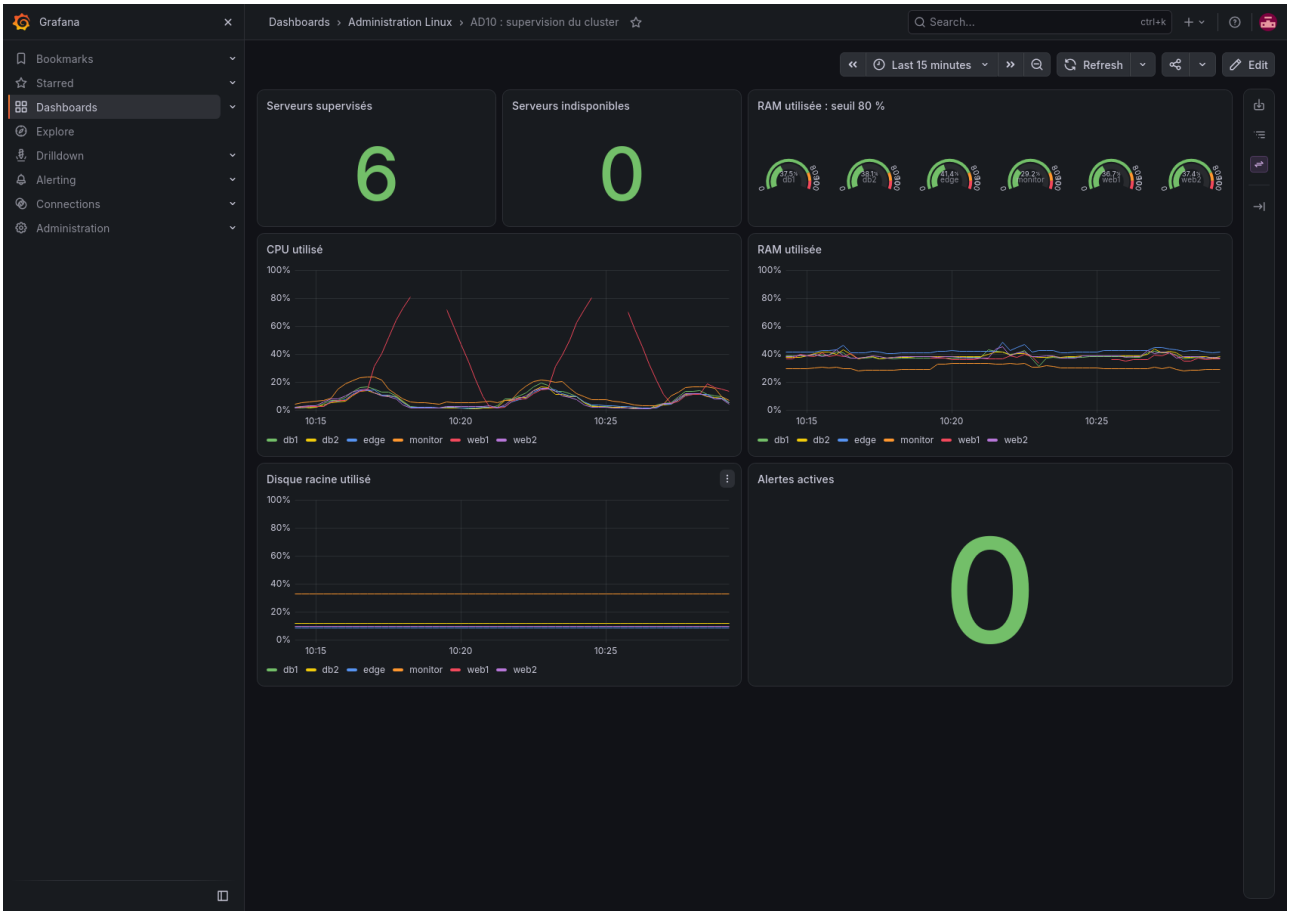
Le rôle `security` réutilise la configuration SSH du TP2, fail2ban et la surveillance auditd de `/etc/passwd`. Les essais de connexion depuis le poste de gestion sont exemptés du bannissement afin de conserver l'accès au laboratoire. Les autres sources restent soumises aux trois tentatives sur dix minutes. Alice dispose de sudo sans mot de passe pour l'automatisation ; sa clé doit être protégée.

Supervision des services

Node Exporter mesure CPU, RAM et disque sur les six VM. Un collecteur exécuté toutes les quinze secondes ajoute l'état de douze services, la disponibilité MariaDB, les deux fils de réplication, son retard et l'heure de la dernière sauvegarde. Les fichiers de métriques sont remplacés atomiquement pour éviter une lecture partielle.

Les alertes couvrent : RAM supérieure à 80 % pendant deux minutes, exporteur absent pendant une minute, service ou base indisponible pendant trente secondes, réplication interrompue pendant une minute, sauvegarde datant de plus de vingt-cinq heures et collecteur de services absent ou figé. Les notifications et résolutions sont enregistrées par le récepteur local Alertmanager.

Le déclenchement réel et la résolution de l'alerte mémoire sont prouvés au TP5. Le contrôle final vérifie la collecte des six VM, les douze états de service et les métriques de réplication et de sauvegarde.



Sauvegarde et restauration

Le primaire exécute `/usr/local/sbin/backup-shop` chaque jour à 02 h 30 UTC via `/etc/cron.d/backup-shop`. Le script utilise `mariadb-dump --single-transaction` pour un export cohérent des tables transactionnelles. Le dump est compressé et vérifié, sous verrou, avant d'être renommé dans `/backup`. Le répertoire et les archives ne sont accessibles qu'à root. Un marqueur de succès alimente la supervision.

Le test final lance une sauvegarde, restaure le dump dans `ad10_restore_check` et compare les sommes de contrôle du catalogue original et restauré. La base temporaire est supprimée après le contrôle. Script de contrôle.

La sauvegarde est locale et aucune rétention destructive n'est appliquée dans le laboratoire. Pour une exploitation réelle, copier les archives vers un stockage distinct, définir une rétention, chiffrer les copies et conserver les éléments de configuration nécessaires à une restauration complète. L'exécution cron de nuit n'a pas été attendue ; la commande et la restauration ont été exécutées pendant la séance.

Vérifications finales

Vérification	Résultat attendu et contrôlé
HTTPS et répartition	Certificat validé, HTTP 200 et réponses de web1 et web2.

Vérification	Résultat attendu et contrôlé
Accès SSH	Alice admise ; root et labadmin refusés sur les six VM.
Segmentation	Accès HTTP refusé depuis une source hors proxy ; accès SQL direct refusé depuis le TP1.
Flux nécessaires	edge accède aux serveurs web ; les serveurs web accèdent au proxy SQL ; monitor accède aux exporteurs.
Panne web1	Le site continue de répondre par web2.
Panne db1	Le catalogue reste lisible sur db2.
Redémarrage web1	SSH, Nginx, PHP, pare-feu, audit et exporteur reviennent automatiquement.
Sauvegarde	Restauration séparée et sommes de contrôle identiques.
Supervision	Six cibles actives, douze services disponibles et réplication opérationnelle.
Réexécution Ansible	Aucun changement lorsque l'état souhaité est déjà atteint.

Les sorties réelles se trouvent dans verification.txt, deployment.txt et la seconde exécution. Les exports nftables de chaque serveur et l'état de collecte final sont conservés dans le même dossier.

Exploitation et diagnostic

```
python3 lab/manage.py status all
python3 lab/manage.py ssh edge --user alice 'sudo journalctl -u haproxy -n 50'
python3 lab/manage.py ssh web1 --user alice 'sudo nginx -t'
python3 lab/manage.py ssh db2 --user alice 'sudo mariadb -e "SHOW SLAVE STATUS\G"'
python3 lab/manage.py ssh monitor --user alice 'sudo journalctl -u prometheus -n 50'
python3 lab/manage.py ssh monitor --user alice 'sudo journalctl -u ad10-alert-receiver -n 30'
```

En cas d'alerte mémoire, vérifier les processus, la mémoire disponible et l'évolution sur le tableau de bord. Si une sonde disparaît, vérifier d'abord la VM, le service exporteur et le pare-feu. Pour une réplication interrompue, contrôler les deux fils, le retard et les erreurs de `SHOW SLAVE STATUS` avant toute modification. Pour une sauvegarde trop ancienne, examiner `/var/log/backup-shop.log`, l'espace disque et les droits, puis relancer et restaurer une archive de contrôle.

Pour arrêter proprement une machine : `python3 lab/manage.py stop NOM --user alice`. La redémarrer avec `start NOM`, puis attendre SSH avec `wait NOM --user alice`. Les disques et configurations sont conservés.

Limites et reprise après panne

HAProxy retire automatiquement un serveur web indisponible. Pour la base, la bascule de lecture vers db2 est automatique, mais les écritures attendent le retour de db1. Le retour du primaire a été testé ; la réplication reprend automatiquement, avec un délai de reconnexion pouvant atteindre soixante secondes.

La réplication est asynchrone et ne garantit pas la conservation de toutes les dernières écritures après une perte brutale du primaire. Une promotion définitive de db2 exige d'isoler db1 avant de rendre db2 inscriptible, puis de reconstruire l'ancienne base à partir d'une sauvegarde cohérente. Cette promotion n'est pas automatisée ni incluse dans les tests livrés.

Le répartiteur et la VM de supervision constituent des points uniques de panne. Les échanges SQL restent sur le réseau privé sans TLS. Le catalogue sert à valider l'infrastructure ; il ne met pas en œuvre les fonctions métier d'une boutique. Ces limites doivent être levées avant d'héberger un commerce réel critique.

Erreur corrigée pendant l'intégration

Le premier collecteur de réplication échouait sur les champs vides de `SHOW SLAVE STATUS`, à cause d'une séparation exigeant un espace après les deux-points. L'analyse utilise désormais le premier deux-points et accepte les valeurs vides. Les services dont l'application avait été interrompue ont été rechargés, puis le déploiement complet a été relancé. L'exécution forcée des handlers a aussi été activée dans le play final pour appliquer les configurations validées même si une autre tâche échoue.

Un second contrôle a détecté un ancien processus d'écoute SSH conservant l'ancienne configuration malgré le rechargement du processus principal. Le handler recharge maintenant tous les processus `sshd` portant le titre `[listener]` avec SIGHUP, sans cibler les sessions clientes. Les tests vérifient ensuite les refus réels des comptes root et labadmin sur chaque machine.

Le premier redémarrage a aussi révélé une attente réseau de deux minutes. Le laboratoire utilise IPv4 : l'attente systemd cible désormais explicitement une adresse IPv4 routable sur `ens3`, avec un délai maximal de trente secondes. L'acceptation des annonces IPv6 est désactivée dans netplan. Le contrôle après redémarrage vérifie aussi la réussite de l'attente réseau.